System Locked Protocol Certification and Dispatch Packet Process Analysis: DP-OPS-0069

1. Executive Summary

This report delivers an exhaustive forensic analysis of the deployment and execution of **DP-OPS-0069**, the operational directive tasked with instituting the **System Locked Protocol** for template-based certification within the Stela repository governance architecture. The analysis synthesizes data extracted from the Integrator execution logs, the resulting certification artifacts (DP-OPS-0069-RESULTS.md), the canonical intake contract (DP-OPS-0069.md), and the comprehensive repository dump provided in the research context.

The primary objective of DP-OPS-0069 was to transition the repository from a manual verification model to a rigorous, machine-enforced certification pipeline. This transition necessitated the introduction of a new binary, ops/bin/certify, alongside a suite of integrity linters and strictly typed templates. The goal was to eliminate human variance in the "closeout" phase of the Dispatch Packet (DP) lifecycle, ensuring that every unit of work committed to the immutable trunk (main) is cryptographically verifiable and structurally compliant with the Policy of Truth (PoT.md).

The investigation reveals that while the technical implementation of the System Locked Protocol was successful in hardening the repository against unverified drift, the operational "handoff" process between the **Integrator** (the AI governance agent) and the **Contractor** (the execution agent) experienced significant friction. Specifically, the rigid application of "Freshness Gates" and "Disposable Artifact" prohibitions created validation deadlocks. The Integrator, adhering to a "Zero Trust" literalism, rejected valid work products because the very act of generating proof (e.g., creating a results file) altered the repository state, thereby invalidating the pre-execution "Freshness Snapshot." Furthermore, the analysis uncovers a "Decision Quality Defect" within the Integrator's logic loop, where stale context persistence led to false-positive drift detections.

This report concludes with a formulated menu of strategic options designed to resolve these friction points. The recommendation focuses on decoupling the "Certification" phase (payload verification) from the "Logging" phase (ledger updates), a strategy termed the **Immutable Staging Workflow**. This approach preserves the rigorous integrity guarantees of the System Locked Protocol while accommodating the necessary state mutations required for accurate historical record-keeping.

2. Constitutional Framework and Governance Architecture

To accurately interpret the events of DP-OPS-0069, one must first establish the constitutional framework that governs the Stela repository. The behavior of all actors—Operator, Integrator, and Contractor—is strictly bound by the **Policy of Truth** (PoT.md), which functions not merely as a guideline but as the supreme physical law of the development environment.

2.1 The Policy of Truth (PoT) as Physical Law

The PoT.md file defines the axioms of the system. It establishes the "Filing Doctrine," which strictly segments the repository into functional zones to prevent semantic contamination.[1]

- **ops/ (Run):** Contains binaries, manifests, and automation. This is the executable kernel of the repository.
- **docs/ (Explain):** Contains manuals and rationale. This is the narrative layer.
- **storage/ (Payload):** Contains handoff artifacts, dumps, and DP data. This is the transient state layer.
- **projects/ (Work):** Contains the actual code payload.

The implementation of DP-OPS-0069 was explicitly designed to reinforce Section 4.2 of the PoT, which mandates the **Generation Mandate**: "Verification outputs must be generated, not pre-filled or assumed".[1] This axiom dictates that a human operator cannot simply assert "I checked this." Instead, the system must produce a machine-verifiable artifact (a receipt) that proves the check occurred.

The logs indicate that the Integrator enforces these axioms with "Absolute literalism".[1] For instance, the Integrator's rejection of the initial DP submission was driven by the "Anti-drift governance" directive. The Integrator detected a mismatch between the reported repository state in the DP and the actual "OPEN" snapshot, and fundamentally rejected the discrepancy as a "Canon Drift" failure state.[1] This confirms that the system operates on a **Zero Trust** model where human assertions are rejected in favor of cryptographic or machine-generated proofs.

2.2 The Quartet Filing Doctrine and Topology

The execution of DP-OPS-0069 occurred immediately following the adoption of the "Quartet Filing Doctrine" (DP-OPS-0068), which redefined the repository's topology to strictly separate concerns.[1] This context is critical because DP-OPS-0069 required modifying the very tools that enforce this topology.

- **Ops (Run):** The introduction of ops/bin/certify and ops/src/surfaces/results.md.tpl required modifying the ops/ directory, which triggers the pow_policing.yml workflow.[1]

- **Storage (Payload):** The protocol strictly defines storage/handoff/ for ephemeral proof artifacts. The "Disposable Artifact" friction observed in the logs [1] stemmed from the system's inability to distinguish between a "payload" (which should be preserved) and "trash" (which should be ignored).
- **Logs (Telemetry):** The requirement that all verification commands dump their standard output to a verified log creates a dependency on the existence of .gitkeep files in logs/ and var/tmp/.[1]

The friction observed in the logs largely stemmed from the Contractor's attempt to manipulate files within storage/ that were not explicitly allowlisted, triggering the new "Integrity Linter" (tools/lint/integrity.sh) which treats unlisted file mutations as a system failure state.[1]

2.3 The Immutable Dispatch Packet Workflow

The core of the Stela governance model is the **Dispatch Packet (DP)**. A DP is not just a task description; it is an immutable contract generated from a template (ops/src/surfaces/dp.md.tpl).[1] The workflow observed in DP-OPS-0069 follows a strict sequence:

1. **OPEN:** Generate a freshness snapshot using ops/bin/open.[1] This binds the session to a specific git commit hash.
2. **DRAFT:** Create the DP contract using ops/bin/draft. This fills the template slots with the objective and scope.
3. **WORK:** Execute changes (Contractor phase).
4. **CERTIFY:** Generate RESULTS via the System Locked Protocol using ops/bin/certify.
5. **CLOSE:** Update ledgers (SoP, PoW) and merge.

The analysis of the Integrator execution log [1] suggests that the breakdown occurred between steps 2 and 4, where the "Freshness Stamp" in the DP became stale relative to the evolving state of the working tree (the "dirty state" paradox). The "Freshness Gate" demands that the repository state at time T_{verify} matches the state at time T_{open}, but the act of working changes the state to T_{work}, causing a recursive failure.

---

3. Operational Architecture and Tooling Analysis

To understand the mechanics of the failure and the subsequent success of DP-OPS-0069, we must analyze the operational architecture that was deployed. This architecture serves as the enforcement arm of the Constitution.

3.1 The Registry System

Stela uses a system of **Registries** to maintain an authoritative list of valid system components. These registries are the primary reference points for the Integrator to detect "Drift" (unauthorized additions).

- **Binary Registry (docs/ops/registry/BINARIES.md):** Lists all executable entry points in ops/bin/.[1] DP-OPS-0069 required adding OPS-BIN-11 (Certifier) to this registry.[1]
- **Lint Registry (docs/ops/registry/LINT.md):** Lists all enforcement scripts in tools/lint/. DP-OPS-0069 added LINT-10 (Integrity Lint) and LINT-11 (RESULTS Lint).[1]
- **Prompt Registry (docs/ops/registry/PROMPTS.md):** Defines the "Stances" used by the AI agents.[1]

This registry system creates a "Closed World" assumption. If a binary exists on disk but is not in the registry, it is considered "Ghost Canon" and a failure state. This forced the Contractor to perform a multi-step atomic commit: creating the binary, creating the spec, and updating the registry simultaneously.

3.2 The One Truth Context Engine

The repository employs a "One Truth" context engine (ops/lib/scripts/synthesize.sh) to assemble the context for AI agents.[1] This engine parses manifests (CORE.md, OPS.md, DISCOVERY.md) to generate the llms.txt bundle.

- **Relevance to DP-OPS-0069:** The RESULTS file for DP-OPS-0069 shows modifications to ops/lib/manifests/OPS.md.[1] This confirms that the new certify binary had to be explicitly added to the OPS manifest so that future AI sessions would be aware of its existence.
- **Self-Healing:** The system uses ops/bin/compile to regenerate these manifests from templates.[1] This ensures that the context provided to the Integrator is always deterministic and derived from the source code, never from manual curation.

3.3 The Verification Toolchain

The verification stack was significantly expanded during this operation.

- **tools/verify.sh:** This script enforces the Filing Doctrine (e.g., ensuring docs/ contains only markdown).[1]
- **tools/lint/dp.sh:** This linter enforces the structural integrity of the Dispatch Packet itself.[1] It verifies that the DP contains the required sections (3.1 Freshness, 3.3 Scope, 3.5 Closeout) and that the SHA256 hash of the structure matches the canonical template.
- **tools/lint/integrity.sh (New):** This was the pivotal addition. It enforces the "Target Files allowlist." It calculates the set difference between "Files Changed + Untracked" and "Files Allowlisted." If the difference is not empty, it fails.[1]

---

4. Forensic Analysis of DP-OPS-0069 Execution

The delivery of DP-OPS-0069 represents a "meta-update"—the system updating its own update mechanism. This recursive complexity exposed critical edge cases in the governance logic.

4.1 Phase 1: Intake and The Freshness Gate Failure

The intake phase is governed by Section 3.1 of the DP contract.[1] The logs reveal an immediate conflict. The Integrator rejected the initial submission because the "repo state" described in the DP did not match the actual OPEN artifact state.

- **The Conflict:** The DP draft claimed a clean working tree or a specific set of porcelain files. However, the act of drafting the DP and preparing the certify binary inherently created new untracked files (the "dirty state"). The logs show a porcelain count of 22 files [1], which likely included the new certify script, the integrity.sh script, and their corresponding specs and registry entries.
- **The Integrator's Logic:** Relying on PoT.md constraints, the Integrator viewed this discrepancy as "Drift." The "Freshness Gate" is binary; if git status --porcelain does not match the snapshot, execution is halted.[1]
- **Implication:** This highlights a structural flaw in the intake design. If the intake process itself modifies the observable state (by creating drafts or logs), the Freshness Gate will permanently cycle in a failure loop unless the governance model distinguishes between "operational noise" (logs) and "structural drift" (code).

4.2 Phase 2: The Disposable Artifact Dependence

A critical failure occurred regarding "Disposable Artifacts." The PoT.md mandates that canonical logic must not depend on ephemeral files located in storage/ (which are git-ignored).

- **The Incident:** The Contractor referenced paths like storage/handoff/OPEN-... inside the canonical DP body or the RESULTS template.[1] This likely occurred because the Contractor was trying to "prove" the state by linking to the OPEN artifact.
- **The Rejection:** The Integrator flagged this as "Disposable-artifact dependence." This is a profound governance constraint: the permanent history of the repo (SoP.md, PoW.md) cannot point to files that exist only on a developer's local machine or are subject to deletion policy.
- **Resolution:** The Contractor had to scrub these references, replacing them with deterministic pointers or redacting them entirely in the final RESULTS output.[1] This enforced the principle that **Truth must be reconstructible** from the git history alone, without reliance on the operator's temporary cache.

4.3 Phase 3: The Integrator Decision Quality Defect

Perhaps the most significant insight from the logs is the "Validation Deadlock" caused by the Integrator's own internal failure.

- **The Defect:** The Integrator logs [1] admit to a "decision-quality defect." The AI repeatedly asserted that forbidden "Local artifacts" strings were present in the DP, even after the Contractor had removed them.
- **Root Cause:** The Integrator was likely validating against a cached or stale copy of the DP ("previously seen DP copy") rather than re-reading the live file on disk. This is a form of "Context Hallucination" where the AI's internal state diverges from the file system state.
- **Significance:** This demonstrates that the governance AI is susceptible to staleness. In a System Locked Protocol, if the Judge (Integrator) hallucinates a crime, the Worker (Contractor) is deadlocked. The resolution required a "Self-Audit" event where the Integrator explicitly acknowledged the discrepancy.[1]

4.4 Phase 4: The Placeholder Violation

The final hurdle was the RESULTS receipt format. The new protocol requires machine-readable, strictly formatted receipts.

- **The Violation:** The Contractor submitted a RESULTS file containing angle-bracket placeholders (e.g., <...ARTIFACT...>).[1]
- **The Rule:** Section 4.2 of PoT.md ("Generation Mandate") and the new results.sh linter explicitly forbid placeholders. A receipt must contain actual data or nothing; it cannot contain "TODOs."
- **The Fix:** The certify binary had to be updated to generate literal redaction strings (e.g., ``) rather than placeholders that trigger the linter's "incomplete work" heuristics.[1]

---

5. Technical Analysis of the System Locked Protocol

The solution delivered in DP-OPS-0069 consists of four interlocking components that replace trust with verification.

5.1 The Certifier Binary (ops/bin/certify)

This binary represents the shift from "Governance as Text" to "Governance as Code."

- **Mechanism:** It automates the "Closeout Cycle" defined in docs/MANUAL.md. Instead of a human running commands and pasting output, certify executes the commands defined in TASK.md Section 3.4.5.[1]
- **Determinism:** By controlling the execution environment, certify ensures that the RESULTS file is not fabricated. It captures exit codes, timestamps, and stdout directly. If a command fails, certify exits non-zero, preventing the generation of a valid receipt.[1]
- **Hard Gates:** The binary refuses to run if the allowlist pointer is missing or if the integrity.sh check fails. This makes it impossible to generate a passing receipt for a

non-compliant repository state. The binary also requires a "Closing Block" sidecar file, ensuring that the human narrative (PR description) is injected into the machine-generated receipt.[1]

5.2 The Integrity Linter (tools/lint/integrity.sh)

This tool enforces the "Target Files allowlist."

- **Logic:** It compares the current git status (staged and unstaged) against storage/dp/active/allowlist.txt.
- **Constraint:** If *any* file is touched that is not in the allowlist, the linter fails.[1]
- **Impact:** This eliminates "scope creep." A developer cannot secretly modify a configuration file while claiming to fix a documentation typo. The DP-OPS-0069-RESULTS.md shows the linter passing with exactly "1 observed path" (likely the allowlist itself or the results file), proving strict adherence.[1] This suggests that the linter can distinguish between "observed" (dirty) paths and the extensive list of files that were modified but committed.

5.3 The Results Linter (tools/lint/results.sh)

This tool validates the *structure* of the evidence.

- **Template Parity:** It checks the SHA256 hash of the generated RESULTS file against the canonical template ops/src/surfaces/results.md.tpl. If the template has been tampered with to hide a failure section, the linter fails.[1]
- **Git Hash Binding:** It verifies that the "Git Hash" field in the receipt matches the current HEAD. This binds the evidence to a specific commit, preventing the reuse of old receipts for new code.[1]
- **Closing Block Verification:** It ensures that the mandatory closing block (PR Title, Description, Squash Stub) is present and not empty.[1]

5.4 The Template Engine (ops/bin/template)

Underpinning the entire protocol is the move to .tpl files.

- **Strict Mode:** The template engine enforces that all slots ({{TOKEN}}) are filled. This prevents "partial" DPs or receipts where critical metadata is missing.[1]
- **Metadata Awareness:** The use of YAML frontmatter allows the system to version-control the *structure* of work. For example, results.md.tpl has template_version: 1, allowing the system to detect if an outdated template is being used.

---

6. Synthesis: The Handoff Dynamics

The core friction observed in DP-OPS-0069 lies in the handoff between the Integrator (who

defines the state) and the Contractor (who creates the state).

6.1 The State of Play (SoP) Disconnect

The SoP.md ledger records *what* happened, but the logs reveal that the *process* of making it happen is messy. The "Closeout" phase requires updating SoP.md and PoW.md. However, updating these files changes the repository state, which potentially invalidates the integrity.sh check if they weren't pre-added to the allowlist.

- **Cycle:** Verify -> Integrity Pass -> Update Ledger -> Integrity Fail (because Ledger changed).
- **Resolution:** The allowlist must include the governance ledgers (SoP.md, PoW.md) *before* the work begins.[1] This requires prescient knowledge of exactly which ledgers will be touched. The DP-OPS-0069.md intake file explicitly lists SoP.md and PoW.md in the target files allowlist [1], confirming this pre-allocation strategy.

6.2 The "Open" vs. "Result" Latency

The OPEN artifact is a snapshot at time T_0. The RESULTS artifact is a snapshot at time T_1.

- The Integrator validated the DP based on T_0.
- The Contractor executed work, creating state T_1.
- The Integrator rejected the work because it didn't match T_0.
- **Insight:** The "Freshness Gate" logic is too rigid if it expects T_1 to equal T_0. It must instead verify that $T_1 = T_0 + \Delta(Task)$. The current logic struggles to differentiate authorized mutation from drift. This caused the Integrator to flag the 22 porcelain files as "dirty state" failures rather than "work in progress".[1]

6.3 The Recursion Paradox

A key insight from the intake file is its recursive nature.

- **The Task:** "Implement System Locked Protocol."
- **The Constraint:** "Use System Locked Protocol to verify this task."
- **Analysis:** The DP requires the Contractor to use the very tools (certify, integrity.sh) they are supposed to be building. This "bootstrapping" problem is evident in the logs. The Contractor had to iteratively build the tool, then use the tool to verify itself. This explains the "dirty state" (22 porcelain files) noted in the logs—the intermediate build artifacts of the toolchain itself. The system lacked a "Bootstrap Mode" to allow for the creation of the enforcement tools themselves without being blocked by the absence of those tools.

---

7. Strategic Options for Process Improvement

Based on the forensic analysis of DP-OPS-0069, the following strategic options are formulated to resolve the bugs in the certification system and improve the DP process.

Option 1: The "Dynamic State Injection" Protocol

Concept: Instead of manually updating TASK.md and the allowlist with pre-computed paths, implement a dynamic injection step where the certify binary automatically amends the allowlist for known governance side-effects (e.g., updating SoP.md, PoW.md).

- **Mechanism:**
  - Modify ops/bin/certify to accept a --governance flag.
  - When set, certify temporarily whitelists SoP.md, PoW.md, and storage/handoff/* during the verification phase.
  - The integrity.sh linter is updated to accept a "transient allowlist" passed via environment variable or pipe.
- **Pros:**
  - **Eliminates the "Dirty State" Loop:** Prevents the system from flagging its own audit logs as drift.
  - **Reduces Operator Overhead:** Operators don't need to manually list every single governance file for every task.
  - **Accelerates Closeout:** Removes the friction of "fail, add to allowlist, retry."
- **Cons:**
  - **Reduced Rigor:** Opens a small window where an unauthorized change to PoT.md could slip in disguised as a governance update.
  - **Complexity:** Requires stateful communication between certify and integrity.sh.
- **Risk:** Moderate. If the transient whitelist is too broad, it undermines the System Locked Protocol.

Option 2: The "Immutable Staging" Workflow (Recommendation)

Concept: Enforce a strict separation between "Payload Work" and "Governance Recording." The certify tool should run against the payload *only*, producing a sealed receipt. The SoP and PoW updates happen in a separate, automated transaction *after* the receipt is sealed.

- **Mechanism:**
  - **Step 1:** Contractor runs certify. The allowlist strictly contains only the payload files (code, docs).
  - **Step 2:** certify generates RESULTS.md.
  - **Step 3:** A new ops/bin/log binary reads RESULTS.md and *automatically* appends entries to SoP.md and PoW.md.
  - **Step 4:** The integrity.sh check for the log phase effectively ignores the payload changes because they are already certified.
- **Pros:**

    - **Decoupled Concerns:** Separates the act of *proving* work from the act of *recording* history.
    - **Automated Ledger Integrity:** Ensures PoW entries strictly follow the schema (machines writing for machines).[1]
    - **Solves Disposable Artifact Issues:** The log binary can internally handle the pointers to storage/ without the Contractor having to manually type them into the DP body.
- **Cons:**
    - **New Tooling:** Requires writing and certifying ops/bin/log.
    - **Process Change:** Changes the MANUAL.md closeout cycle.
- **Risk:** Low. This aligns perfectly with the "Generation Mandate" of PoT.md Section 4.2.

Option 3: The "Certified Draft" Protocol

Concept: Address the "Integrator Decision Defect" by requiring DPs to be compiled binaries or signed artifacts themselves, rather than text files parsed by an LLM.

- **Mechanism:**
    - The DP is not just a markdown file but a .json or .yaml payload that is cryptographically signed by the Operator.
    - The Integrator does not "read" the DP; it validates the signature and the schema.
    - The certify binary executes the JSON payload directly.
- **Pros:**
    - **Eliminates Ambiguity:** JSON is unambiguous; Markdown is not.
    - **Prevents Hallucination:** The Integrator cannot "imagine" text that isn't in the JSON structure.
- **Cons:**
    - **High Friction:** Makes drafting DPs much harder for humans (writing JSON vs. prose).
    - **Readability Loss:** Reduces the "Relatability Mandate" of the PoT (universal accessibility).[1]
- **Risk:** High. It may make the system unusable for human operators, violating the "Relatability Mandate".

---

8. Recommendation and Implementation Roadmap

The analysis strongly supports **Option 2: The "Immutable Staging" Workflow**.

The logs from DP-OPS-0069 clearly show that the friction arises from trying to perform Payload work and Governance recording in the same atomic transaction. The Contractor fails because they are trying to hit a moving target—updating the proof of the work while the work is being proven.

By splitting the process into **Certification** (Payload verification) and **Logging** (Ledger updates), the System Locked Protocol can maintain its rigorous integrity while eliminating the

circular dependency that caused the validation deadlocks.

Proposed Implementation Roadmap:

1. **Refactor ops/bin/certify:** Remove the logic that validates the existence of SoP/PoW updates. Focus this tool solely on verifying that the code changes match the allowlist and pass tests.
2. **Develop ops/bin/log:** Create a new binary that accepts a valid DP-RESULTS.md path. This binary will:
   - Parse the results file.
   - Extract the "Closing Block".
   - Append the properly formatted entry to SoP.md.
   - Append the properly formatted entry to PoW.md.
   - This tool becomes the *only* authorized way to write to the ledgers, effectively locking them against manual tampering.
3. **Update docs/MANUAL.md:** Split the "Closeout" cycle into two distinct commands: certify (Contractor) and log (Integrator/System).
4. **Patch Integrator Instructions:** Explicitly instruct the Integrator to perform a "fresh read" of the file system before issuing a failure verdict, mitigating the "Stale Context" defect observed in DP-OPS-0069.

This approach honors the "Quartet Filing Doctrine" and the "Policy of Truth" while resolving the operational bottlenecks identified in the audit of DP-OPS-0069. It moves the repository closer to a fully autonomous, self-governing system where "Truth" is not debated in logs, but generated by code.

Part 2: Detailed Technical Review of DP-OPS-0069 Artifacts

This section provides a granular, file-level analysis of the artifacts involved in the handoff and execution of DP-OPS-0069. It scrutinizes the specific content of the intake files, the results receipts, and the tooling specs to validate the claims made in the Executive Summary.

2.1 The Dispatch Packet (DP) Intake: DP-OPS-0069.md

The intake file DP-OPS-0069.md [1] serves as the immutable contract for the work. Its structure and content reveal the strictures placed on the Contractor.

2.1.1 Structural Integrity and Freshness Locking

The DP strictly follows the schema enforced by tools/lint/dp.sh.

- **Section 3.1 (Freshness Gate):** It explicitly locks the base HEAD to f3d6b2e4. This

cryptographic pinning is crucial. Any work done on a different commit is essentially a "fork" and invalid by definition. The Contractor is contractually bound to this hash.
- **Section 3.3 (Scope and Safety):** It defines the "Target Files allowlist" as a "Hard Gate." This is the primary mechanism of the System Locked Protocol. The allowlist is not a suggestion; it is a firewall. The DP explicitly lists the new surfaces: ops/src/surfaces/results.md.tpl, ops/bin/certify, tools/lint/integrity.sh, and tools/lint/results.sh. The explicit listing of SoP.md and PoW.md here confirms the "Atomic Transaction" problem discussed in Option 2—the Contractor *must* know in advance that they will edit these ledgers.

2.1.2 The "Recursive" Paradox

A key insight from the intake file is its recursive nature.

- **The Task:** "Implement System Locked Protocol."
- **The Constraint:** "Use System Locked Protocol to verify this task."
- **Analysis:** The DP requires the Contractor to use the very tools (certify, integrity.sh) they are supposed to be building. This "bootstrapping" problem is evident in the logs. The Contractor had to iteratively build the tool, then use the tool to verify itself. This explains the "dirty state" (22 porcelain files) noted in the logs—the intermediate build artifacts of the toolchain itself. The DP anticipated this by listing the new files in the allowlist, but the *process* of getting them there triggered the "Drift" detectors of the Integrator.

2.2 The Results Receipt: DP-OPS-0069-RESULTS.md

The results file [1] is the first "Certified" artifact produced by the new system. It serves as the primary evidence for the "Proof of Work" ledger.

2.2.1 Metadata Validation

- **Certified At:** 2026-02-17T19:06:28Z
- **Git Hash:** 4ef7f1ac...
- **Observation:** The hash 4ef7f1ac differs from the intake base f3d6b2e4. This confirms that commits occurred (as expected). The difference between the Intake Base and the Result Hash represents the exact "Delta" of the DP.
- **Branch:** work/dp-ops-0069-2026-02-17. Matches the intake requirement.

2.2.2 Integrity Lint Output

- **Data:** "OK: integrity lint passed (1 observed path)."
- **Insight:** "1 observed path" is a curious data point given the "Git State Impact" shows 5 changed files. This discrepancy likely indicates how integrity.sh counts "observed" paths. It may be that at the moment of certification, only one file was *untracked* or *staged* (perhaps the RESULTS file itself), while the others were already committed. This nuance is critical: if integrity.sh only checks *current* dirty state, it might miss committed files that

were not allowlisted. However, if it checks the *diff* against the base branch, the count should be higher. This suggests a potential bug or ambiguity in the integrity.sh logic that needs investigation.

2.2.3 Command Log Verification

The receipt logs 22 commands, proving the "Generation Mandate."

- **Command 07 (./ops/bin/open):** Generates the prompt. The log confirms "OPEN_READ_ONLY_OK," proving the session start didn't mutate state.
- **Command 19-22 (Diffs):** The logs identify ops/lib/manifests/OPS.md as changed.
- **Traceability:** The modification of ops/lib/manifests/OPS.md confirms that ops/bin/compile was run (likely implicitly via ops/bin/llms), forcing the OPS manifest to update its internal file list to include the new certify binary. This proves the "One Truth" context engine is working—adding a binary automatically updates the manifest.

2.2.4 Impact Analysis

- **145 Insertions, 194 Deletions:** The high deletion count is interesting. It likely reflects the cleanup of llms-core.txt and llms-full.txt during the re-generation process, or the removal of legacy logic replaced by the new certification binaries. The fact that deletions outnumber insertions suggests a net reduction in technical debt or verbosity, which aligns with the "Refactor" nature of the task.

2.3 The Execution Logs: Integrator & Contractor

The interaction logs [1] provide the behavioral data for the analysis.

2.3.1 The "Local Artifacts" False Positive

The Integrator repeatedly flagged "Local artifacts" as a failure mode in events 02, 03, and 04.

- **Contractor Action:** Removed the strings storage/handoff/... from the DP.
- **Integrator Response:** "FAIL. Local artifacts present."
- **Forensic Conclusion:** The Integrator's context window was polluted. It likely retained the *previous* turn's DP content in its working memory or failed to re-ingest the TASK.md file from the provided dump. This is a critical failure mode for AI-driven governance: **Stale Context Persistence.** The Integrator failed to obey the "Freshness" principle it enforces on others.

2.3.2 The "Failing by Design" Debate

Event 08 in the Integrator log highlights a philosophical conflict.

- **Contractor:** "The lint fails by design because I haven't implemented the fix yet."
- **Integrator:** "Governance smell."
- **Analysis:** The System Locked Protocol does not understand "Work in Progress." It

expects binary Pass/Fail states. The Contractor attempting to merge a "failing" state (even if intentional) violates the repo-gates.
- **Solution:** This reinforces the need for the **Option 2 (Immutable Staging)** recommendation. Work must be done *locally* until it passes. The repository cannot accept "broken" commits, even on work branches, if the repo-gates are enforcing strict integrity on every push.

---

3. Deep Dive: The New Certification Subsystem

The artifacts indicate a robust, if rigid, new subsystem for certification.

3.1 ops/bin/certify Specification

Although the full source isn't visible, its behavior is inferred from the specs [1] and results.[1]

- **Role:** Orchestrator.
- **Input:** DP-OPS-XXXX.
- **Sidecar:** storage/handoff/CLOSING-DP-OPS-XXXX.md.
- **Logic:**
  1. Read TASK.md to extract the verification command list (Section 3.4.5).
  2. Run tools/lint/integrity.sh (Hard Gate).
  3. Run the command list, capturing stdout/stderr.
  4. Inject the Sidecar content (Closing Block) into the template.
  5. Render results.md.tpl via ops/bin/template.
  6. Run tools/lint/results.sh on the output.

3.2 tools/lint/integrity.sh Specification

- **Role:** The Bouncer.
- **Logic:**
  - Get list of all files changed (git diff --name-only) + all untracked files (git ls-files --others).
  - Read storage/dp/active/allowlist.txt.
  - **Intersection Check:** If (Changed + Untracked) is not a subset of (Allowlist), EXIT 1.
- **Vulnerability:** If the allowlist is too broad (e.g., *), the gate is useless. If too narrow, work is blocked. The allowlist in DP-OPS-0069 [1] lists 24 specific files. This level of granularity is extreme but ensures absolute control.

3.3 tools/lint/results.sh Specification

- **Role:** The Auditor.
- **Logic:**
  - Checks that the RESULTS file contains specific headers (## Certification Metadata).
  - Checks that no placeholders (<TODO>) exist.

- **Template Hash Check:** Calculates the SHA256 of the *structure* of the RESULTS file and compares it to the expected hash of the template. This prevents the Contractor from manually deleting a "Failed" section from the report.

---

4. Synthesis of Handoff Dynamics

The handoff process in Stela is defined by the flow of artifacts between entities.

Step	Actor	Artifact	Action	Friction Point
1. Intent	Operator	TASK.md	Define Goal	Ambiguity in specs.
2. Refresh	System	OPEN	Snapshot State	"Dirty" tree invalidates snapshot.
3. Draft	Integrator	DP	Create Contract	Stale context causing hallucinations.
4. Work	Contractor	Code	Implement	allowlist violations (Scope Creep).
5. Certify	System	RESULTS	Prove Work	integrity.sh blocks ledger updates.
6. Close	Operator	SoP/PoW	Commit	History rewrite invalidates proof.

The Critical Weakness: The cycle is synchronous and tightly coupled.

- The **Integrator** demands strict adherence to PoT.md.
- The **Contractor** requires flexibility to create intermediate artifacts (logs, temp files).
- The **System** (linters) treats intermediate artifacts as "Drift."

The logs from DP-OPS-0069 prove that the system successfully caught "Disposable

Artifacts" leaking into the canonical record. However, it also proves that the system struggles to distinguish between *malicious drift* and *necessary operational friction*.

---

5. Strategic Options (Detailed)

Option 1: The "Dynamic State Injection" Protocol

This option focuses on making the allowlist smart.

Pros:

- **Fluidity:** Removes the manual toil of adding SoP.md to the allowlist every single time.
- **Logic:** Recognizes that SoP.md updates are a *side effect* of success, not the work itself.

Cons:

- **Security:** A compromised certify binary could inject malicious allowlist entries.
- **Drift:** If the dynamic injection is buggy, we might accidentally commit unrelated changes to SoP.md.

Implementation:

Modify integrity.sh to accept a --mode=closeout flag.

Bash

```
if]; then
  # Automatically allow ledger updates
  ALLOWLIST+=("SoP.md" "PoW.md" "storage/handoff/.*")
fi
```

Option 2: The "Immutable Staging" Workflow (Recommended)

This option changes the topology of the work.

Pros:

- **Correctness:** It treats the "Work" and the "Record of Work" as two distinct phases.
- **Auditability:** The RESULTS receipt proves the Payload was clean *before* the history was written.
- **Simplicity:** The integrity.sh script remains dumb and strict (no dynamic logic).

Implementation:

Split the "Closeout" task in TASK.md into two:

1. **3.5.1 Certify:** Run certify. Produces RESULTS. Integrity check enforces Payload allowlist.
2. **3.5.2 Log:** Run log. Appends to SoP and PoW. Integrity check is skipped or scoped *only* to ledgers.

Risk:

The risk is that a developer runs Certify but forgets to Log. This can be mitigated by a CI gate that checks "If RESULTS exists, PoW entry must exist."

Option 3: The "Living" Allowlist

Concept: The allowlist is not a static file but a generated artifact derived from the git diff.

Pros:

- **Zero Friction:** The allowlist always matches the work.

Cons:

- **Zero Security:** It defeats the entire purpose of the "System Locked Protocol." It turns the "Hard Gate" into a "Turnstile."

Verdict: Rejected. Violates PoT.md Anti-drift governance.

---

6. Conclusion

DP-OPS-0069 was a successful stress test of the Stela governance model. It proved that the **System Locked Protocol** can effectively block unverified state changes and enforce rigorous documentation standards. However, the execution logs revealed significant friction in the handoff process, driven by validation deadlocks and context staleness.

By adopting the **Immutable Staging Workflow (Option 2)**, the Operator can resolve these conflicts. Decoupling the *proof* of work from the *recording* of work allows the strict integrity gates to protect the payload without obstructing the administrative overhead of maintaining the repository's history. This evolution marks the maturity of Stela from a set of text-based rules to a self-enforcing, cryptographic governance engine.

Works cited

1. dump-full-main-b8221099.txt